Printer thanks joe

Spruce version 11.2 -- spooler version 11.2

File: oz:<kmp>sig.lisp.28

Creation date: 12 November 1984 22:59:45

For: KMP

6 total sheets = 5 pages, 1 copy.

```
;;; -*- Mode: LISP; Package: ERR,GLOBAL,1000; Base: 10; Fonts:CPTFONTB; -*-

;(signal condition-type
;       :condition-arg-name1 condition-arg1
;       :condition-arg-name2 condition-arg2 ...
;   case1
;   case2
;   case3 ...)
```

```
(DEFMACRO +THROW (TG VL)
  '(*THROW ,TG (MULTIPLE-VALUE-LIST ,VL)))

(DEFMACRO +CATCH (TG &BODY FORMS)
  '(PROG T ()
         (RETURN-FROM T (VALUES-LIST (*CATCH ,TG
                                             (RETURN-FROM T (PROGN ,@FORMS)))))))


(DEFVAR *CONDITION-HANDLERS* '())

(DEFVAR *ACTIVE-CONDITION* NIL)

(DEFUN *DECLINE (CONDITION)
  (IF (EQ CONDITION *ACTIVE-CONDITION*)
      (+THROW 'DECLINE NIL)
    (ERROR "Can't decline ~S" CONDITION)))

(DEFUN DECLINE (CONDITION)
  (SEND CONDITION ':DECLINE))

(DEFUN *DISMISS (CONDITION)
  (IF (EQ CONDITION *ACTIVE-CONDITION*)
      (+THROW 'DISMISS NIL)
    (ERROR "Can't dismiss ~S" CONDITION)))

(DEFUN DISMISS (CONDITION)
  (SEND CONDITION ':DISMISS))

(DEFUN PROCEED? (CONDITION TYPE &REST ARGS)
  (IF (AND (EQ CONDITION *ACTIVE-CONDITION*)
           (MEMQ TYPE (SEND CONDITION ':PROCEED-TYPES)))
      (+THROW 'PROCEED (LEXPR-FUNCALL #'VALUES TYPE ARGS)))
  NIL)

(DEFUN PROCEED (CONDITION TYPE &REST ARGS)
  (OR (LEXPR-FUNCALL #'PROCEED? CONDITION TYPE ARGS)
      (ERROR "The proceed option ~S isn't valid." TYPE)))

(DEFUN HANDLE-SIGNAL (CONDITION)
  (LET ((*ACTIVE-CONDITION* CONDITION))
    (+CATCH 'DISMISS
      (PROG HANDLE-SIGNAL ()
            (DO ((H *CONDITION-HANDLERS* (CDR H)))
                ((NULL H)
                 (RETURN-FROM HANDLE-SIGNAL
                   (+CATCH 'DECLINE
                     (+THROW 'DISMISS
                             (LET ((*CONDITION-HANDLERS* '()))
                               (SEND CONDITION ':HANDLE-DEFAULT))))))
              (WHEN (TYPEP CONDITION (CAR (CAR H)))
                (+CATCH 'DECLINE
                  (RETURN-FROM HANDLE-SIGNAL
                    (+CATCH 'PROCEED
                      (LET ((*CONDITION-HANDLERS* (CDR H)))
                        (FUNCALL (CADR (CAR H)) CONDITION))
                      (ERROR
                        "Handler did not DISMISS, DECLINE, or PROCEED: ~S"
                        CONDITION)))))))))

(DEFMACRO SIGNL (CONDITION-NAME &BODY BODY)
  (LET ((ARGS (NCONS CONDITION-NAME))
        (CLAUSES BODY))
    (DO ()
        ((OR (NULL CLAUSES)
             (NOT (ATOM (CAR CLAUSES)))))
      (PUSH (POP CLAUSES) ARGS)
```

(+THROW 'PROCEED
(LEXPR-FUNCALL
(SEND CONDITION :ESCAPE TYPE)
ARGS))

```
      (PUSH (POP CLAUSES) ARGS))
    (LET ((KEY (GENSYM))
          (TEMPS (DO ((C CLAUSES (CDR C))
                      (X '()))
                     ((NULL C)
                      (MAPCAR #'(LAMBDA (IGNORE) (GENSYM)) X))
                   (IF (AND (NOT (ATOM (CAAR C)))
                            (> (LENGTH (CDAAR C))
                               (LENGTH X)))
                       (SETQ X (CDAAR C))))))
      (LET ((FORM '(*SIGNAL ,@(NREVERSE ARGS)
                            ':PROCEED-TYPES ',(MAPCAR #'(LAMBDA (CLAUSE)
                                                          (IF (ATOM (CAR CLAUSE))
                                                              (CAR CLAUSE)
                                                              (CAAR CLAUSE)))
                                                      CLAUSES))))
        (IF (NOT CLAUSES) FORM
          (LET ((BODY-FORM
                 '(SELECTQ ,KEY
                    ,@(MAPCAR #'(LAMBDA (CLAUSE)
                                  (COND ((ATOM (CAR CLAUSE))
                                         '((,(CAR CLAUSE)) ,@(CDR CLAUSE)))
                                        (T
                                         '((,(CAAR CLAUSE))
                                           ,@(IF (CDAR CLAUSE)
                                                 '((LET ,(MAPCAR #'LIST
                                                                 (CDAR CLAUSE)
                                                                 TEMPS)
                                                     ,@(CDR CLAUSE)))
                                                 (CDR CLAUSE))))))
                              CLAUSES))))
            (IF TEMPS '(MULTIPLE-VALUE-BIND (,KEY ,@TEMPS)
                                            ,FORM ,BODY-FORM)
              '(LET ((,KEY ,FORM)) ,BODY-FORM)))))))

(DEFUN *SIGNAL (CONDITION-NAME &REST KEYWORD-ARGS)
  (LET ((CONDITION (LEXPR-FUNCALL #'MAKE-INSTANCE CONDITION-NAME KEYWORD-ARGS)))
    (HANDLE-SIGNAL CONDITION)))

(DEFMACRO LET-CONDITIONS (CLAUSES &BODY FORMS)
  (IF (NOT CLAUSES) '(PROGN ,@FORMS)
    '(LET ((*CONDITION-HANDLERS* (APPEND
                                   (LIST ,@(MAPCAR #'(LAMBDA (X)
                                                       '(LIST ',(CAR X) ,(CADR X)))
                                                   CLAUSES))
                                   *CONDITION-HANDLERS*)))
       ,@FORMS)))
```

```
(COMMENT

(DEFFLAVOR C (PROCEED-TYPES) () :SETTABLE-INSTANCE-VARIABLES)
(DEFMETHOD (C :DEFAULT :HANDLE-DEFAULT) () NIL)
(DEFMETHOD (C :DISMISS) () (*DISMISS SELF))
(DEFMETHOD (C :DECLINE) () (*DECLINE SELF))
(DEFFLAVOR FOO (X Y Z) (C) :SETTABLE-INSTANCE-VARIABLES)
(DEFFLAVOR FOOP (X Y Z) () :SETTABLE-INSTANCE-VARIABLES)
(DEFFLAVOR BAR (A B C) (C) :SETTABLE-INSTANCE-VARIABLES)
(DEFFLAVOR BAZ (D E F) (C) :SETTABLE-INSTANCE-VARIABLES)
(DEFFLAVOR DEBUG () (C))
(DEFMETHOD (DEBUG :HANDLE-DEFAULT) ()
  (DBG)
  (DISMISS SELF))
(DEFFLAVOR FOOBAR () (FOO BAR))
(DEFFLAVOR FOOBAZ () (FOO BAZ))
(DEFFLAVOR FOOD () (FOO DEBUG))
(DEFFLAVOR FOODP () (FOOP DEBUG))


(LET-CONDITIONS ((FOOBAZ #'(LAMBDA (CONDITION)
                             (PROCEED? CONDITION 'ZING 17.)
                             (DISMISS CONDITION))))
  (LET-CONDITIONS ((FOO #'(LAMBDA (CONDITION) (DECLINE CONDITION)))
                   (BAR #'(LAMBDA (CONDITION) (DISMISS CONDITION)))
                   (BAZ #'(LAMBDA (CONDITION) (IF (Y-OR-N-P "Dismiss? ")
                                                  (DISMISS CONDITION)
                                                  (DECLINE CONDITION)))))
    (SIGNL 'FOOBAZ :X 3 ((ZAP A B) (LIST 'B= B 'A= A)))))

)
```

```
;;; Local Modes:
;;; Lisp WITH-DECLINE-ALLOWED-FOR Indent:1
;;; Lisp WITH-DISMISS-ALLOWED-FOR Indent:1
;;; Lisp RETURN-FROM Indent:1
;;; Lisp BLOCK Indent:1
;;; End:
^@^@
```